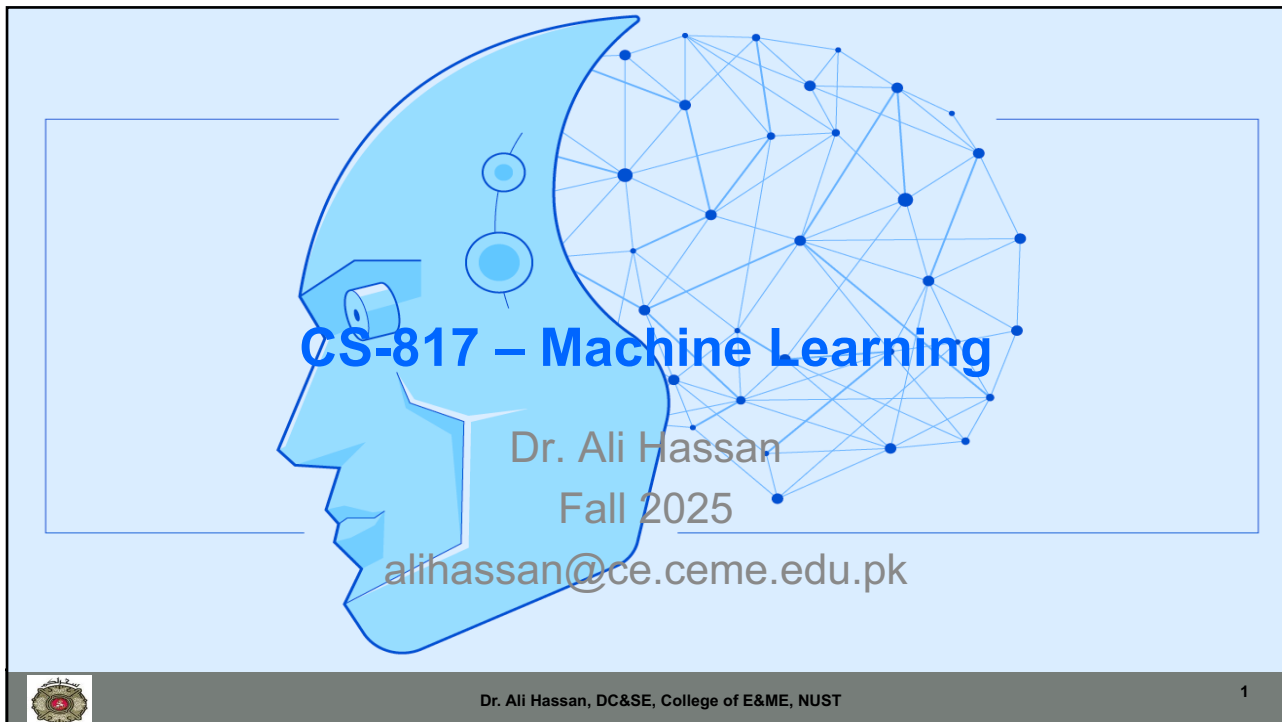


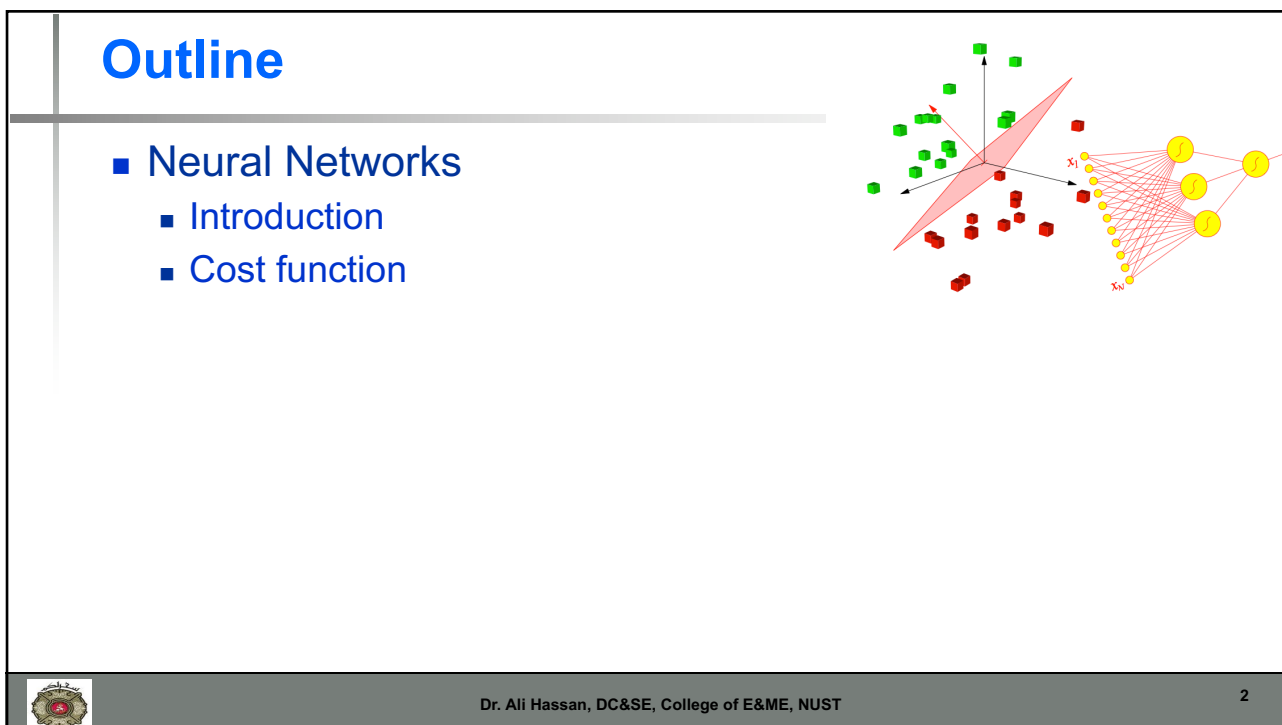


CS-817 – Machine Learning

Dr. Ali Hassan
Fall 2025
alihassan@ce.ceme.edu.pk

Dr. Ali Hassan, DC&SE, College of E&ME, NUST

1



Outline

- Neural Networks
 - Introduction
 - Cost function

Dr. Ali Hassan, DC&SE, College of E&ME, NUST

2

Non-linear hypotheses

NEURAL NETWORKS



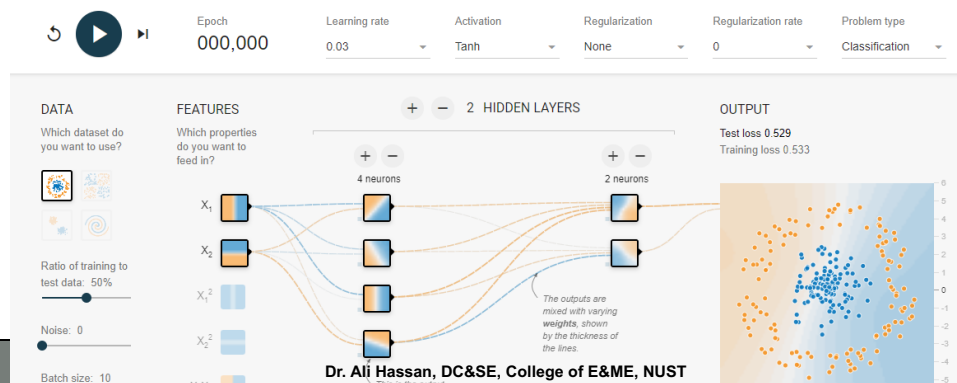
3

3

Lets play with neural networks

- <http://playground.tensorflow.org/>

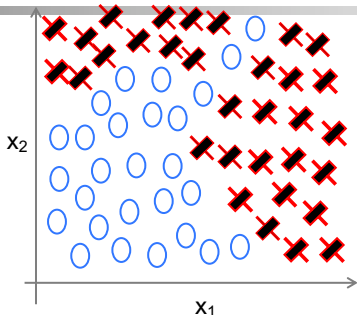
Tinker With a **Neural Network** Right Here in Your Browser.
Don't Worry, You Can't Break It. We Promise.



4

4

Non-linear Classification



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^3 x_2 + \theta_6 x_1 x_2^2 + \dots)$$

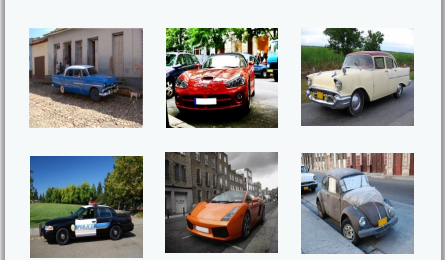
x_1 = size
 x_2 = # bedrooms
 x_3 = # floors
 x_4 = age
...
 x_{100}



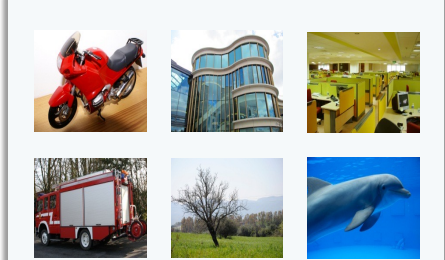
5

5

Computer Vision: Car detection



Cars



Not a car

Testing:



What is this?



6

6

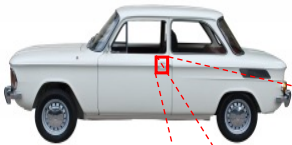
<https://k4all.org/project/pascal2/>



7

What is this?

You see this:

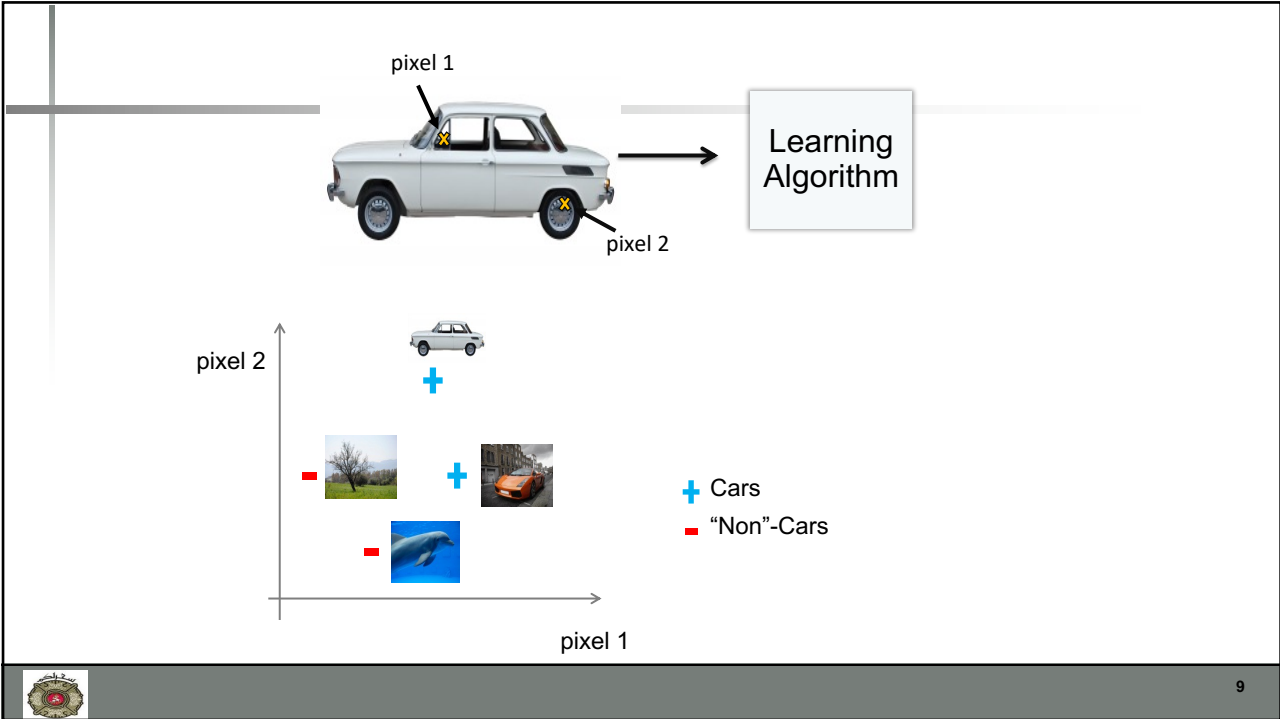


But the camera sees this:

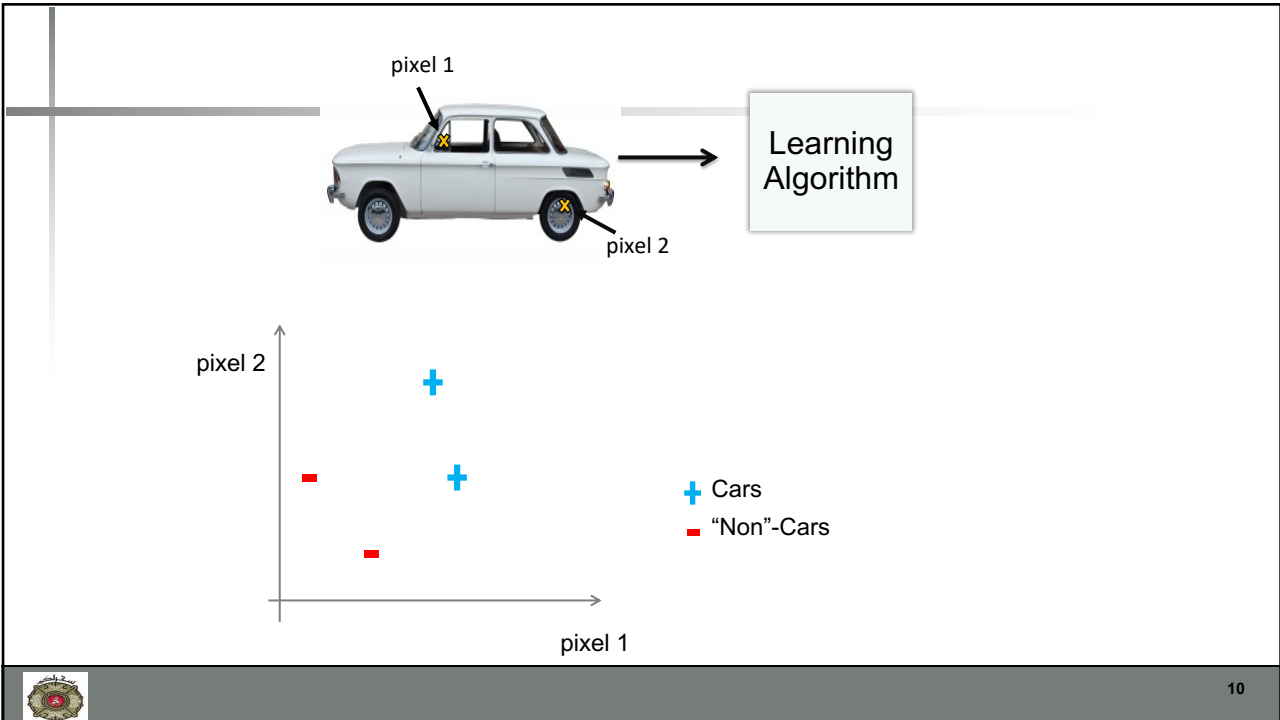
194	210	201	212	199	213	215	195	178	158	182	209
180	189	190	221	209	205	191	167	147	115	129	163
114	126	140	188	176	165	152	140	170	106	78	88
87	103	115	154	143	142	149	153	173	101	57	57
102	112	106	131	122	138	152	147	128	84	58	66
94	95	79	104	105	124	129	113	107	87	69	67
68	71	69	98	89	92	98	95	89	88	76	67
41	56	68	99	63	45	60	82	58	76	75	65
20	43	69	75	56	41	51	73	55	70	63	44
50	50	57	69	75	75	73	74	53	68	59	37
72	59	53	66	84	92	84	74	57	72	63	42
67	61	58	65	75	78	76	73	59	75	69	50



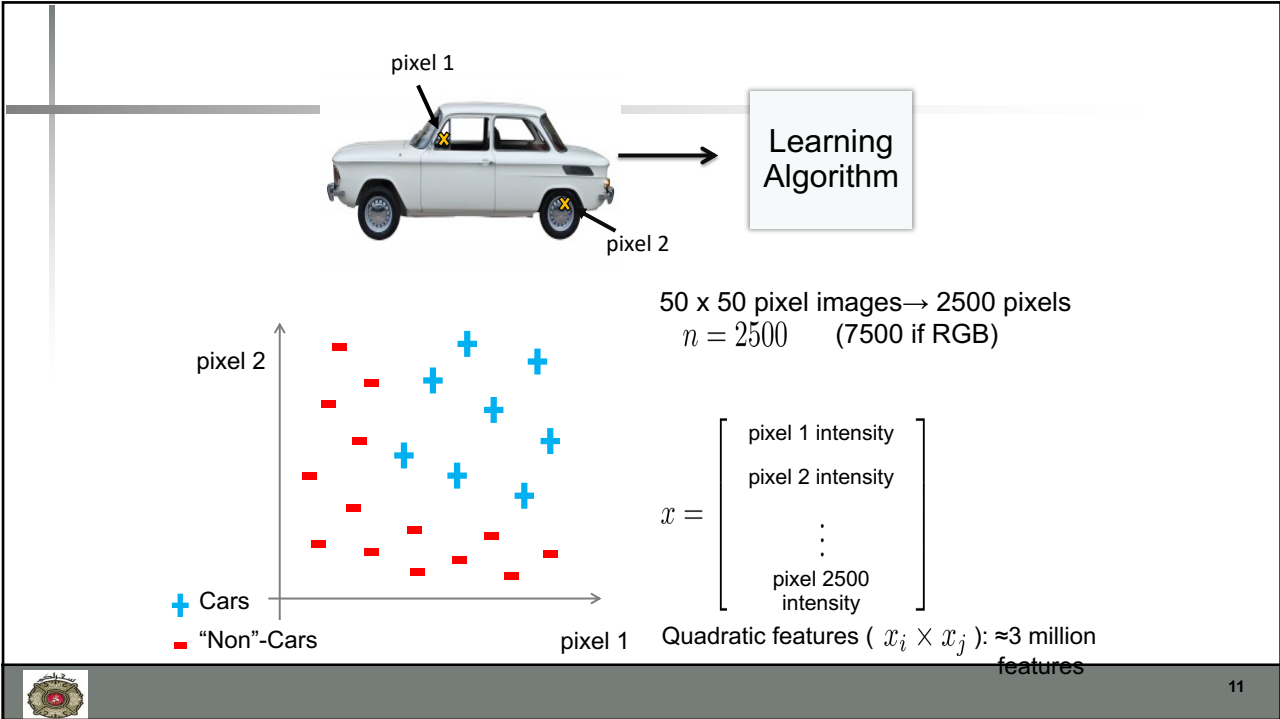
8



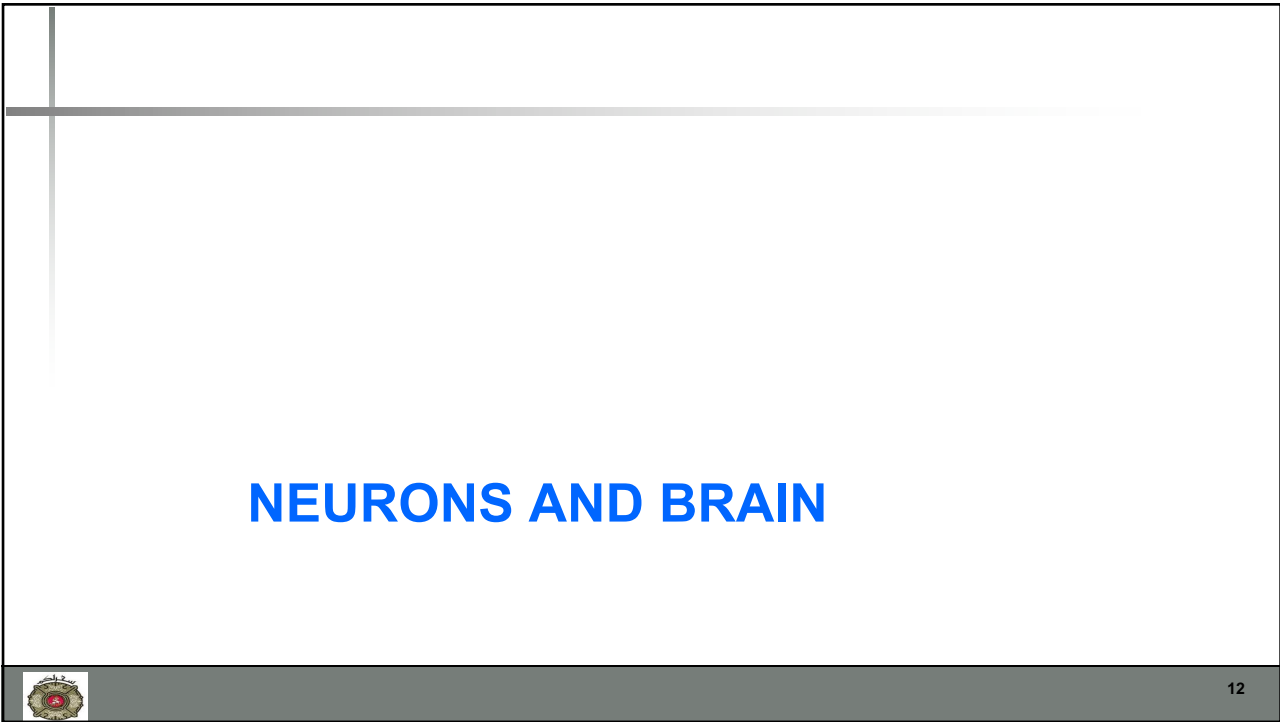
9



10



11



12

Hubel and Wiesel Cat Experiment



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

13

13

Neural Networks

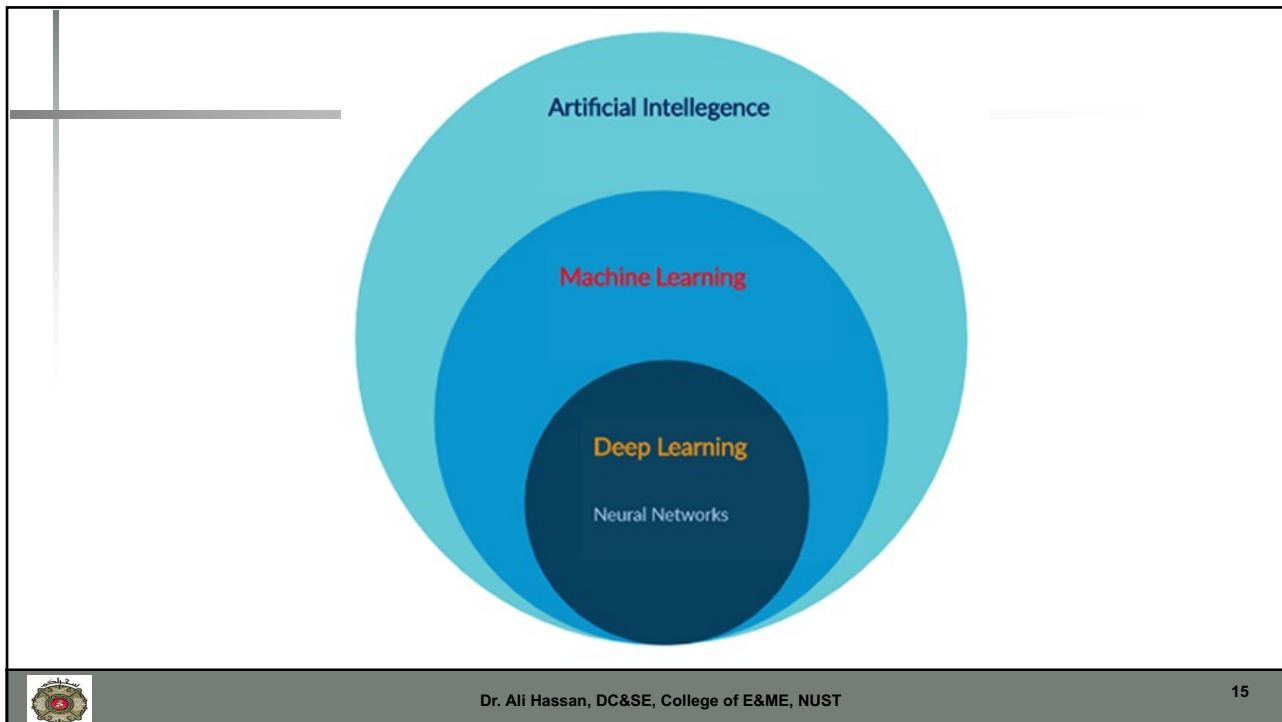
- Origins: Algorithms that try to mimic the brain.
- Was very widely used in 80s and early 90s; popularity diminished in late 90s.
- Recent resurgence under the name of **Deep Neural Networks**: State-of-the-art technique for many applications



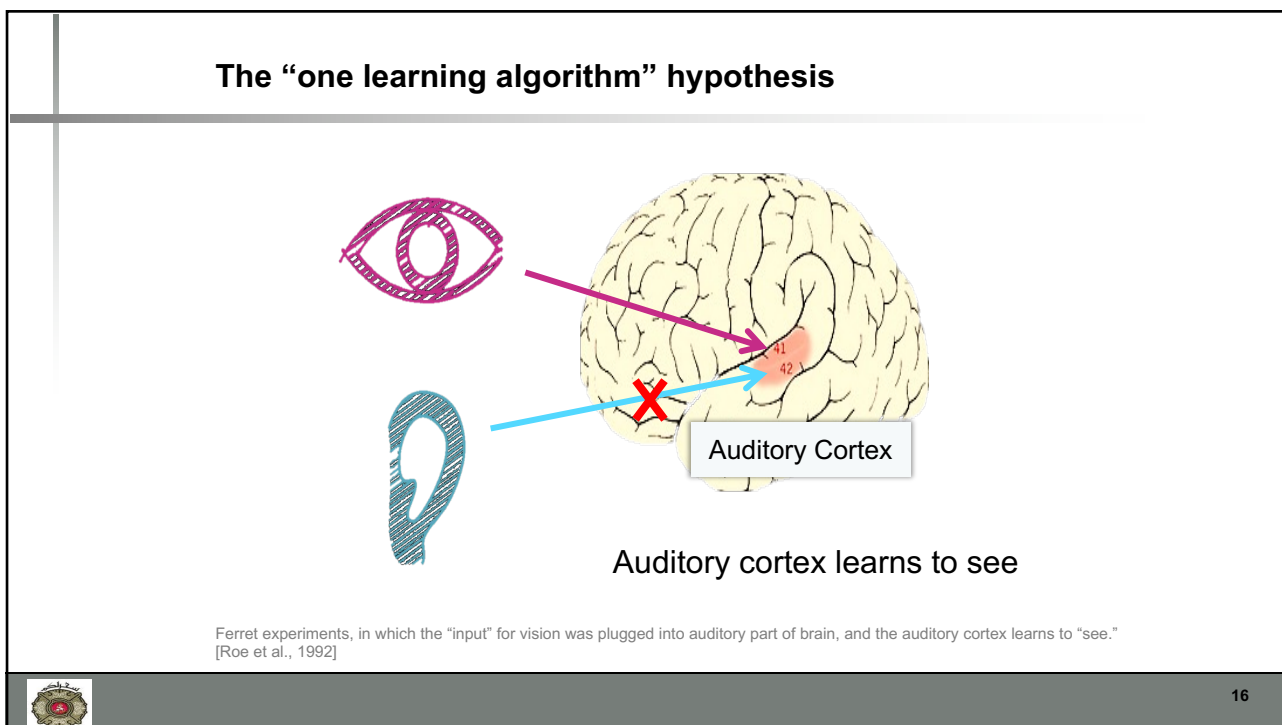
Dr. Ali Hassan, DC&SE, College of E&ME, NUST

14

14

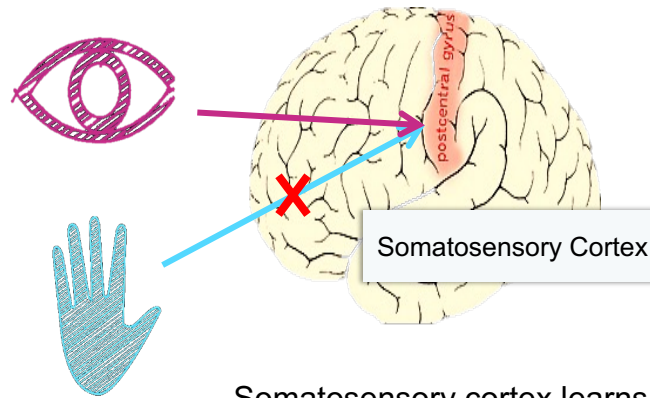


15



16

The “one learning algorithm” hypothesis



Somatosensory cortex learns to see

[Metin & Frost, 1989]



17

17

Daniel Kish: Human Echo locator



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

18

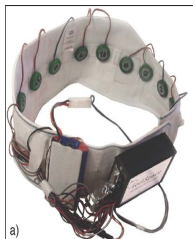
18

Sensor representations in the brain



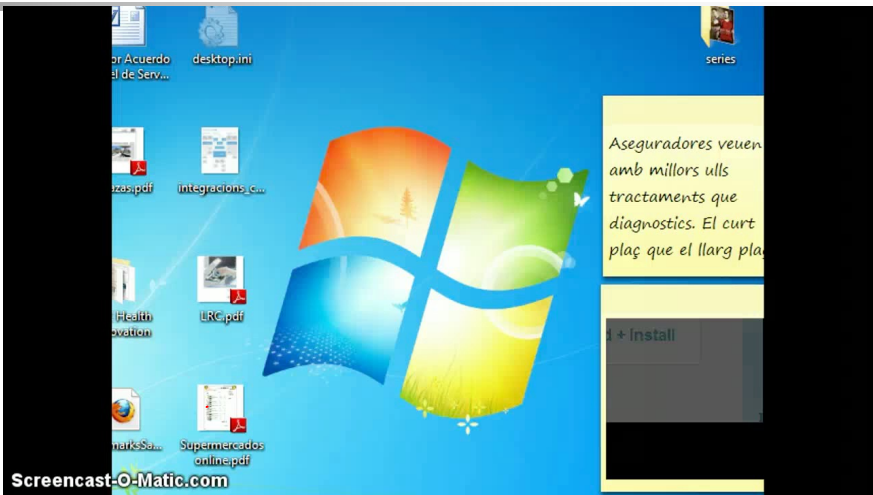
Seeing with your tongue

Human echolocation (sonar)



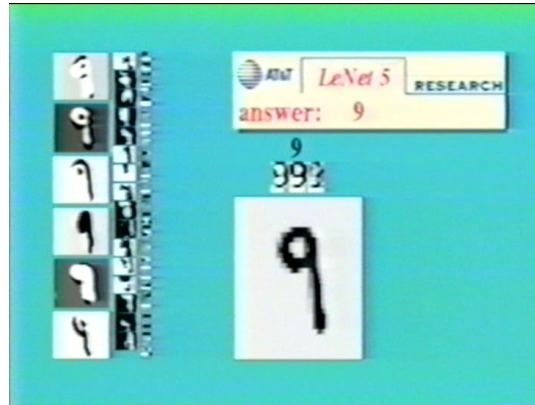
Haptic belt: Direction sense

Implanting a 3rd eye



[Courtesy of Dean Pomerleau]





21

21

Neural Networks

MODEL REPRESENTATION



22

22

Brain Differences

- before you start thinking that you can just create a huge neural network and call strong AI
- A human brain has **100 billion neurons** and **100 trillion connections** and operates on 20 watts (enough to run a dim light bulb)
- In comparison the biggest neural network have **10 million neurons** and **1 billion connections** on 16,000 CPUs (about 3 million watts)



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

23

23

Brain Differences

- The brain is limited to **5 types of input** data from the **5 senses**
- Children do not learn what a cow is by reviewing 100,000 pictures labelled “cow” and “not cow”, but this is how machine learning works.
- Probably we don't learn by calculating the **partial derivative** of each neuron related to our initial concept. (By the way we don't know how we learn)



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

24

24

Take home message

- There is still a lot of **room for improvement**
- Hence
- There is still a lot of **room for research**

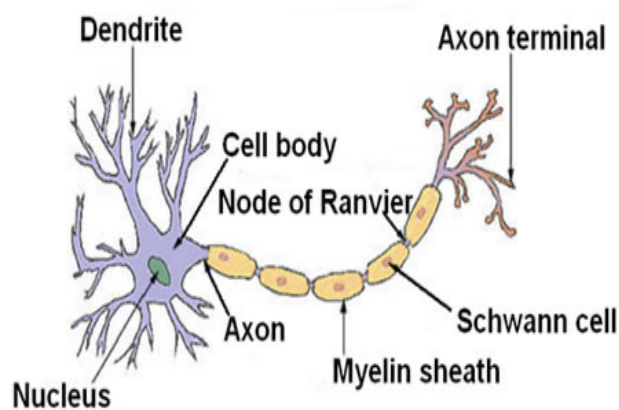


Dr. Ali Hassan, DC&SE, College of E&ME, NUST

25

25

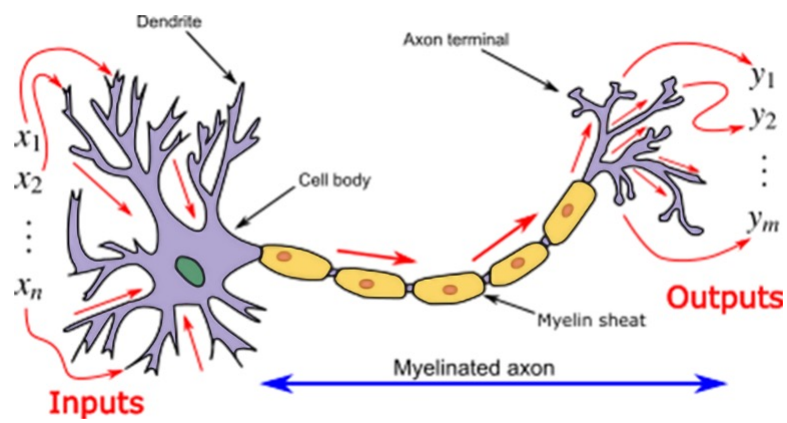
Neuron in the brain



26

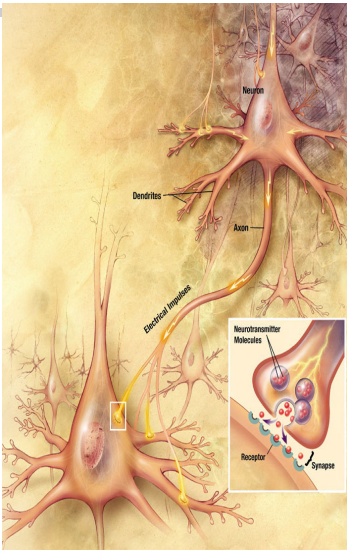
26

Neural Network Representation



27

Neurons in the brain



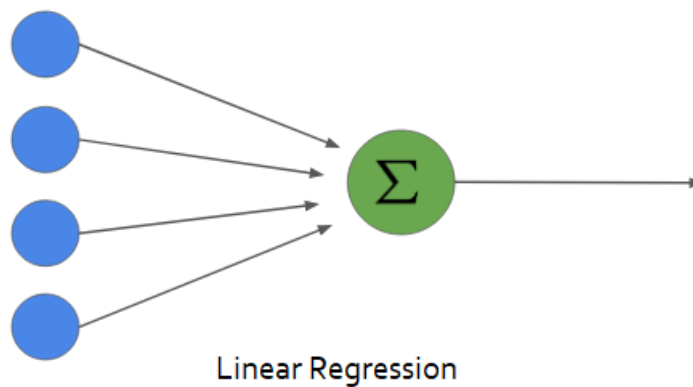
28

EXAMPLE FROM PREVIOUS LECTURES



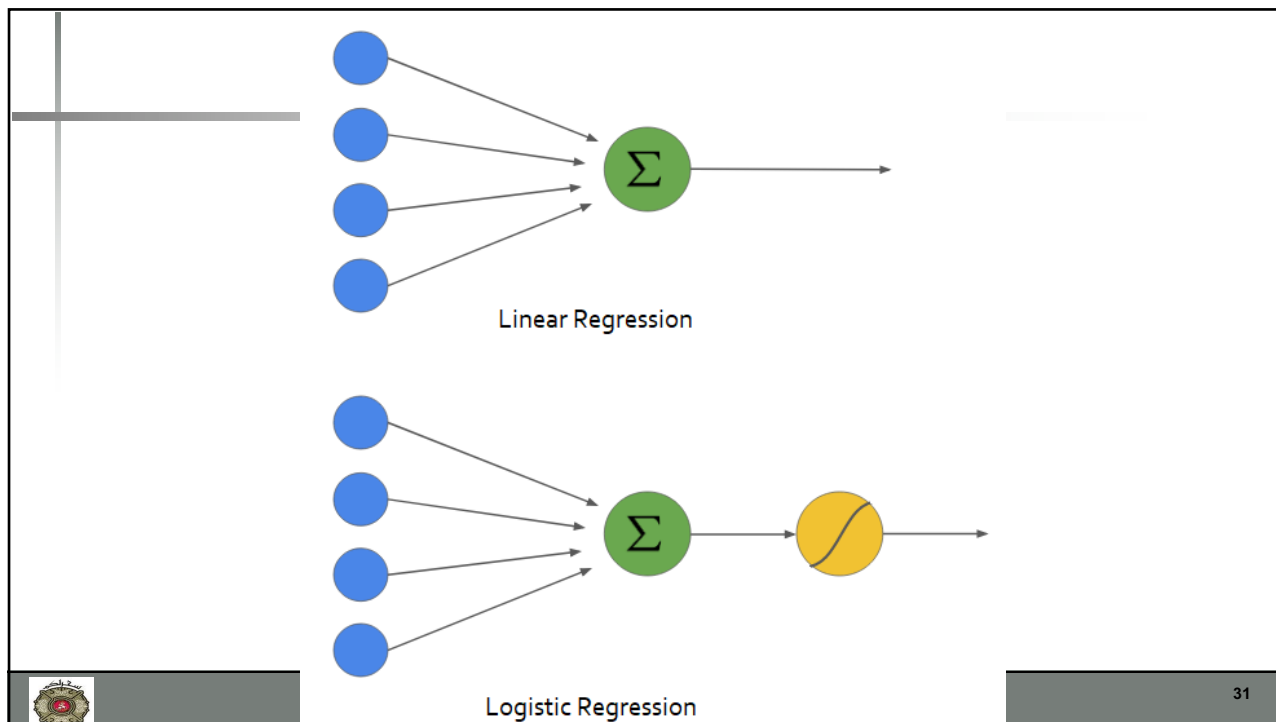
29

29



30

30



31

Example – house on fire

- Suppose you wake up in the middle of night, your house is on fire.
- What will your priorities be in this case:
 - Your beloved ones
 - Essential documents, degrees etc
 - Jewelry
 - Laptop
 - A pencil

**We assign
priority to
each**

32



What is a Neural Network

- A NN or ANN represents **interconnected input and output units** in which each connection has an **associated weight**
- During the learning phase, the network **learns by adjusting these weights** in order to be able to predict the correct class for input data



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

33

33

Applications of ANN

- Classification of data:
 - Based on a set of data, our trained neural network predicts whether it is a dog or a cat?
- Anomaly detection:
 - Given the details about transactions of a person, it can say that whether the transaction is fraud or not
- Speech recognition:
 - We can train our neural network to recognize speech patterns. Example: Siri, Alexa, Google assistant.



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

34

34

Applications of ANN

- Audio generation:
 - Given the inputs as audio files, it can generate new music based on various factors like genre, singer, and others.
- Time series analysis:
 - A well trained ANN can predict the **stock price**.
- Spell checking:
 - We can train a ANN that detects misspelled spellings and can also suggest a similar meaning for words. Example: **Grammarly**



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

35

35

Applications of ANN

- Character recognition:
 - A well trained ANN can detect **handwritten** characters.
- Machine translation:
 - We can develop a ANN that translates one language into another language. **Google Translate**
- Image processing:
 - We can train a neural network to process an image and extract pieces of information from it.

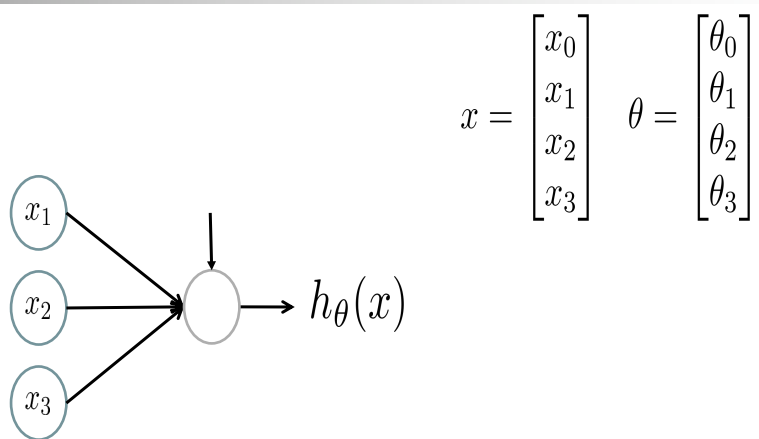


Dr. Ali Hassan, DC&SE, College of E&ME, NUST

36

36

Neuron model: Logistic unit



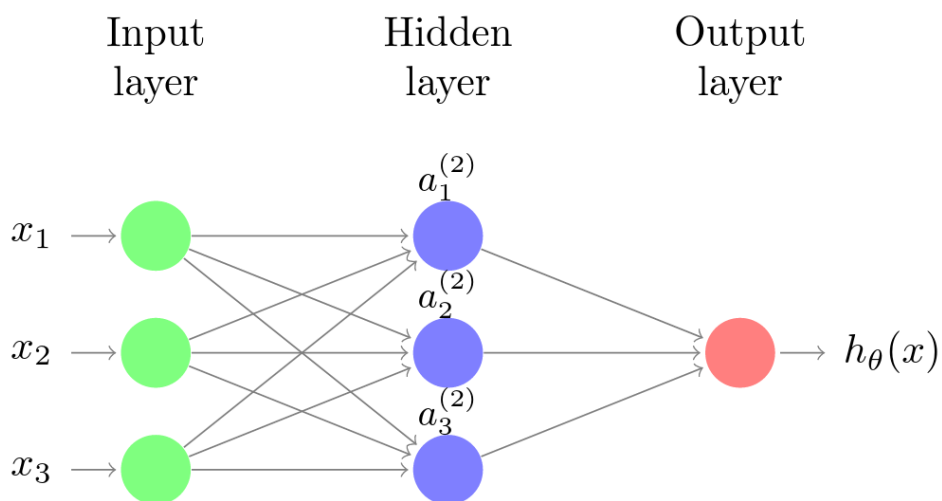
Sigmoid (logistic) activation function.



37

37

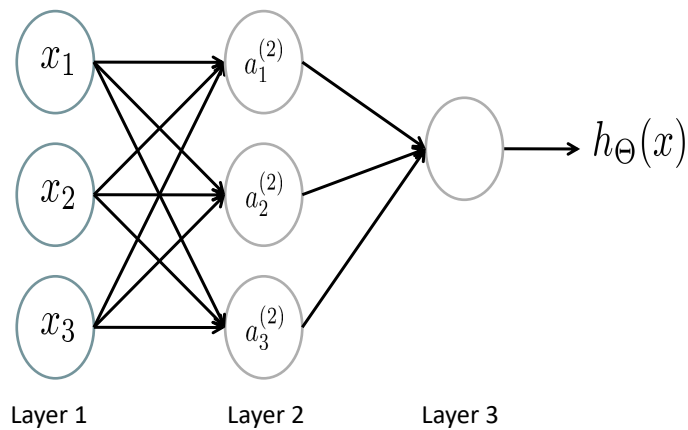
Example of simple NN



38

38

Neural Network



39

39

Sigmoid Function

```
# Import required libraries :
import numpy as np
import matplotlib.pyplot as plt

# Generating input dataset :
# It will generate 100 points between -10 and 10.

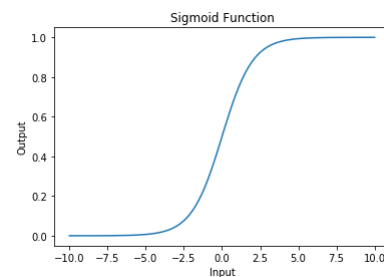
input = np.linspace(-10,10,100)

# Define sigmoid function :

def sigmoid(X):
    val = 1/(1+np.exp(-X))
    return val

# Output values :
output = sigmoid(input)

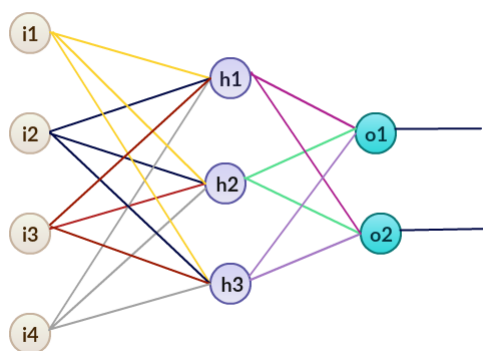
# Plotting the graph for sigmoid function :
plt.plot(input,output)
plt.xlabel("Input")
plt.ylabel("Output")
plt.title("Sigmoid Function")
```



40

40

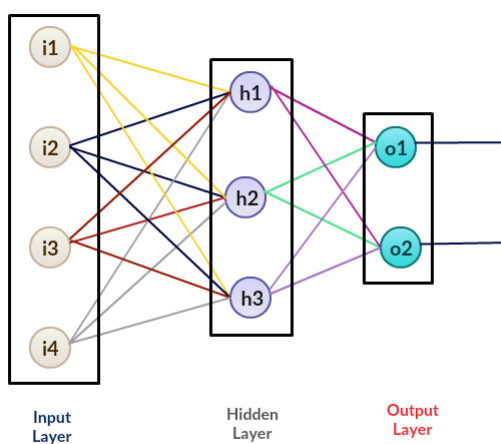
General Structure of ANN:



41

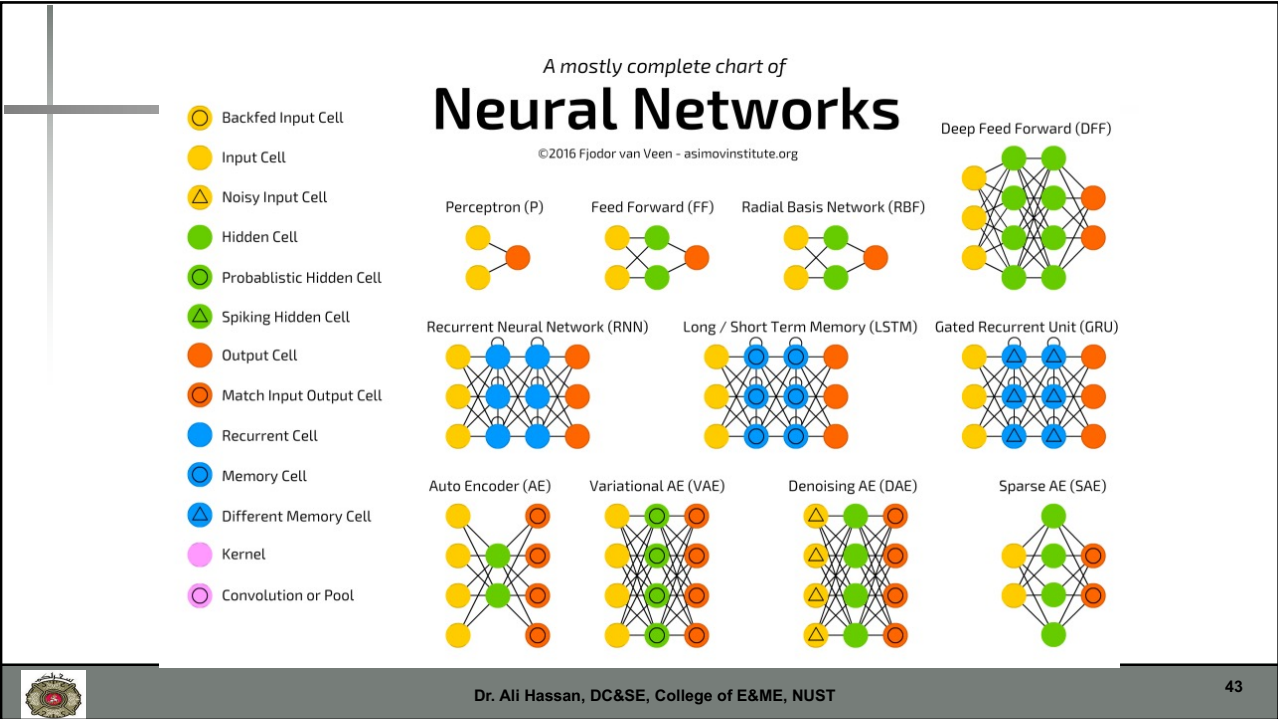
41

General Structure of ANN:

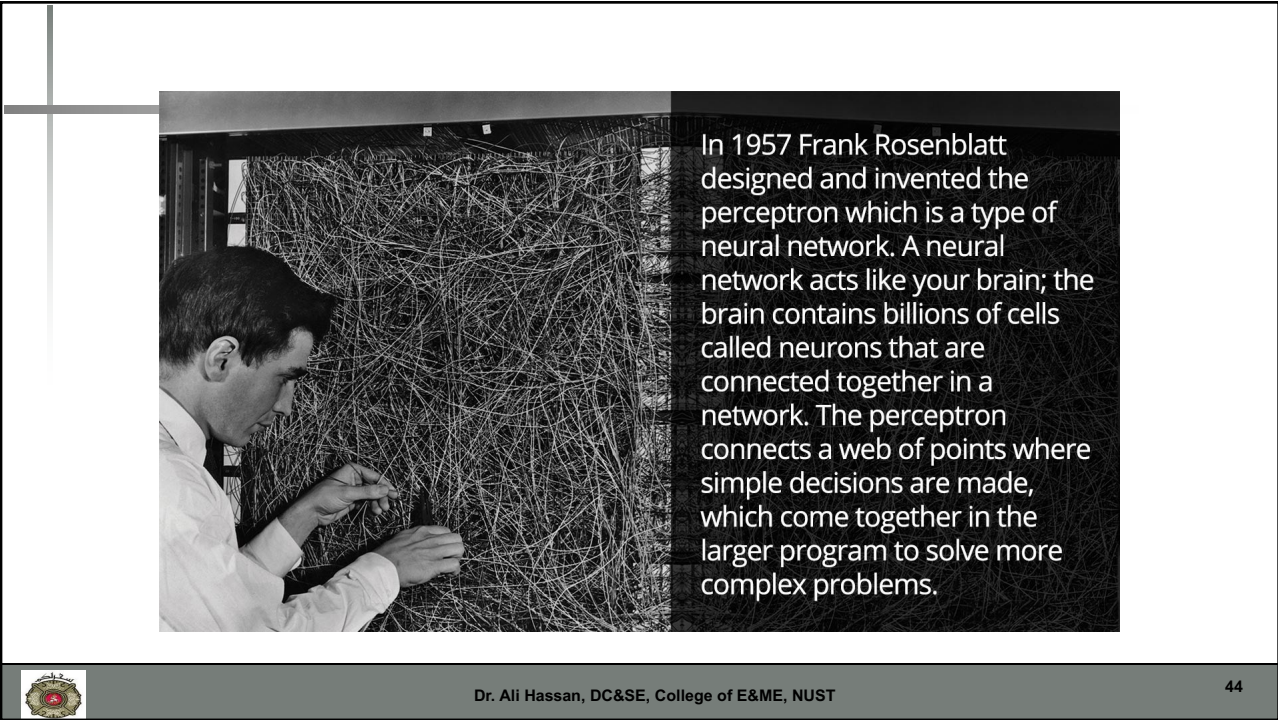


42

42



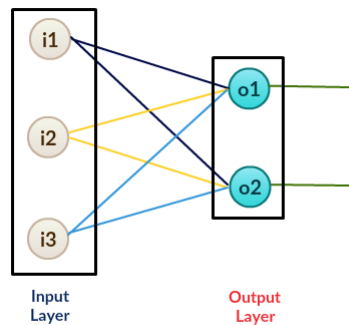
43



44

What is a Perceptron

- A perceptron is a neural network **without** any **hidden layer**.
- A perceptron only has an **input layer** and an **output layer**.



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

45

45

Steps for implementation of NN

1. Feedforward:
 - On a feedforward neural network, we have a set of **input features** (x_i) and some random weights (w_i or θ_i). Notice that in this case, we are taking random weights that we will optimize using backward propagation.
2. Backpropagation:
 - During backpropagation, we calculate the **error** between predicted **output** and **target** output and then use an algorithm (gradient descent) to update the weight values.

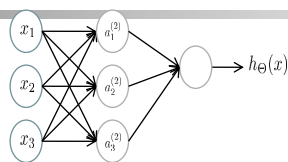


Dr. Ali Hassan, DC&SE, College of E&ME, NUST

46

46

Neural Network



$a_i^{(j)}$ = "activation" of unit i in layer j

$\Theta^{(j)}$ = matrix of weights controlling function mapping from layer j to layer $j + 1$

$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$

$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$

$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

$$h_{\Theta}(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

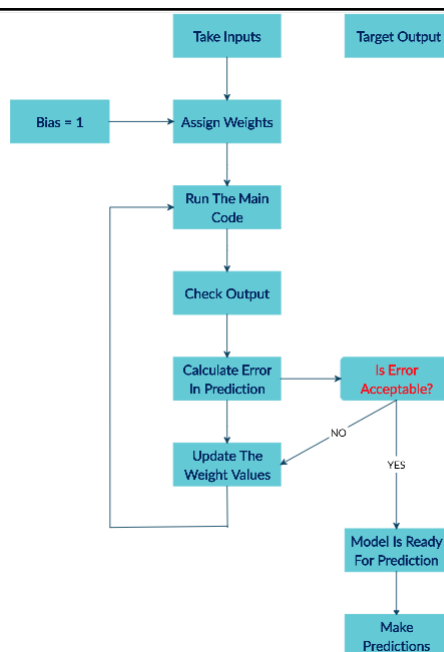
If network has s_j units in layer j , s_{j+1} units in layer $j + 1$, then $\Theta^{(j)}$ will be of dimension $s_{j+1} \times (s_j + 1)$.



47

47

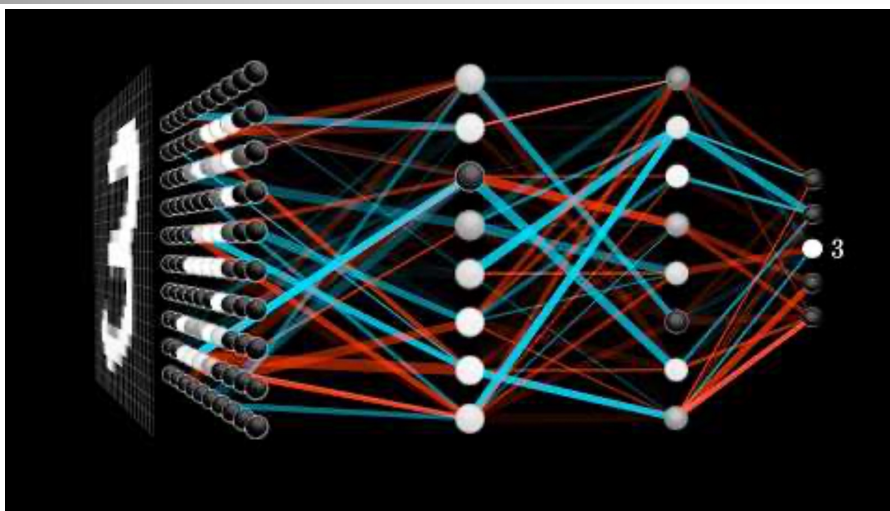
Flow chart f



48

48

What is Neural Network



https://youtu.be/aircAruvnKk?si=GsmBshoupLYtU_1k



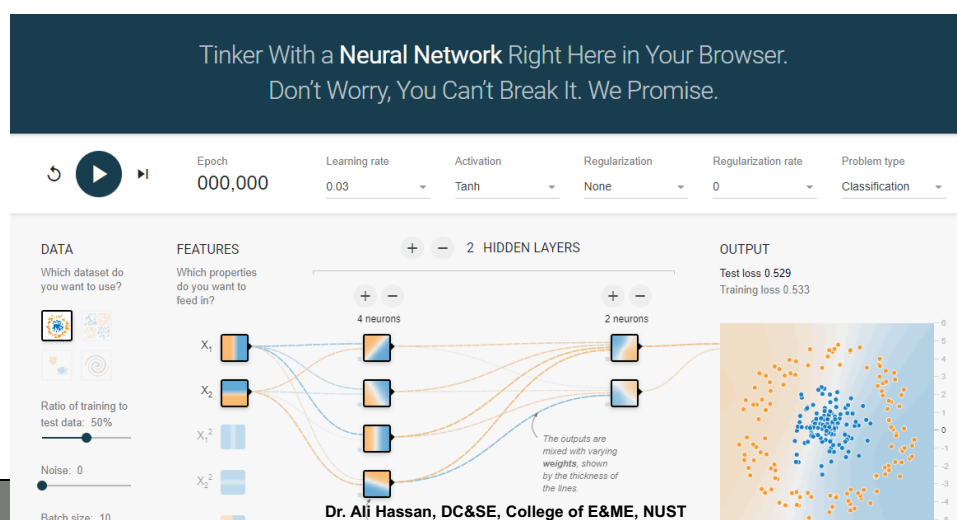
Dr. Ali Hassan, DC&SE, College of E&ME, NUST

49

49

Lets play with neural networks

- <http://playground.tensorflow.org/>



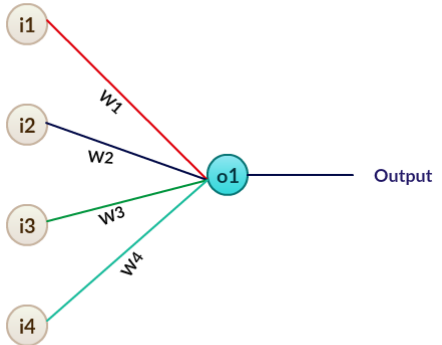
Dr. Ali Hassan, DC&SE, College of E&ME, NUST

50

50

Perceptron Example

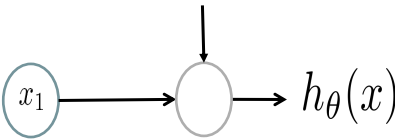
- In this example we are going to calculate the output by the following formula:
- $Output = \theta_o + \theta_1x_1 + \theta_2x_2 + ... + \theta_nx_n$
- θ_i = weights
- x_i = inputs



Input	Target Output
0	0
1	2
2	4
3	6



Perceptron Example



Input	Target Output
0	0
1	2
2	4
3	6



Perceptron Example

- For the sake of this example, we are going to take the bias value = 0 for simplicity of calculation
- Let's take $\theta = 3$ and check the predicted output

Input	Target Output	Predicted Output (W=3)
0	0	0
1	2	3
2	4	6
3	6	9



Perceptron Example

Input	Target Output	Predicted Output (W=3)	Error In Prediction
0	0	0	0
1	2	3	1
2	4	6	2
3	6	9	3



Perceptron Example

Input	Target Output	Predicted Output (W=3)	Error In Prediction	Predicted Output (W=4)
0	0	0	0	0
1	2	3	1	4
2	4	6	2	8
3	6	9	3	12

Input	Target Output	Predicted Output (W=3)	Error In Prediction (W=3)	Predicted Output (W=4)	Error In Prediction (W=4)
0	0	0	0	0	0
1	2	3	1	4	2
2	4	6	2	8	4
3	6	9	3	12	6



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

55

55

Perceptron Example

Input	Target Output	Predicted Output (W=2)
0	0	0
1	2	2
2	4	4
3	6	6

Input	Target Output	Predicted Output (W=2)	Error In Prediction (W=2)
0	0	0	0
1	2	2	0
2	4	4	0
3	6	6	0

■ Global Minima achieved



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

56

56

Perceptron Example – Summary

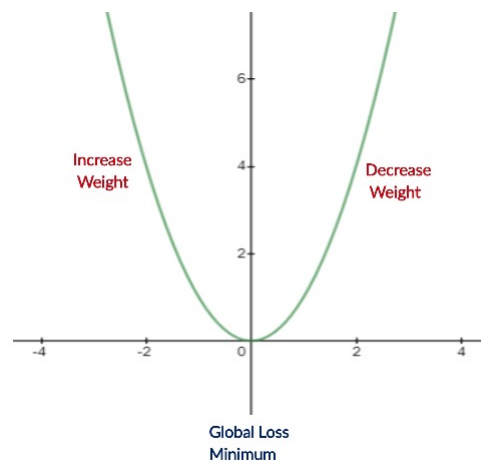
- Now what we did here
 - First, we have our input values and target output.
 - Then we initialized some random value to θ , and then we proceed further.
 - Last, we calculated the error for in prediction for that weight value. Afterward, we updated the weight and predicted the output. After several trial and error epochs, we can reduce the error in prediction.



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

57

57



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

58

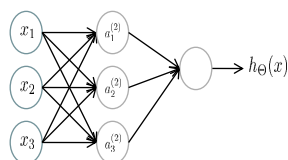
58

Neural Networks

MODEL REPRESENTATION II

59

59

Forward propagation: Vectorized implementation

$$a_1^{(2)} = g(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3)$$

$$a_2^{(2)} = g(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3)$$

$$a_3^{(2)} = g(\theta_{30}^{(1)} x_0 + \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{33}^{(1)} x_3)$$

$$h_{\Theta}(x) = g(\theta_{10}^{(2)} a_0^{(2)} + \theta_{11}^{(2)} a_1^{(2)} + \theta_{12}^{(2)} a_2^{(2)} + \theta_{13}^{(2)} a_3^{(2)})$$

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix}$$

$$z^{(2)} = \Theta^{(1)} x$$

$$a^{(2)} = g(z^{(2)})$$

$$\text{Add } a_0^{(2)} = 1$$

$$z^{(3)} = \Theta^{(2)} a^{(2)}$$

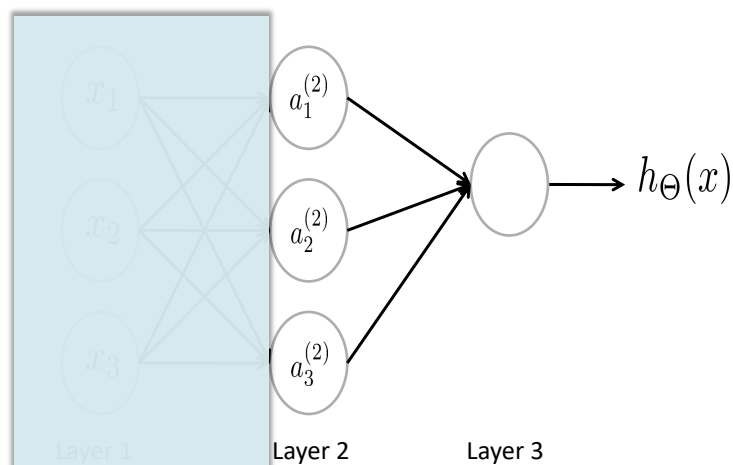
$$h_{\Theta}(x) = a^{(3)} = g(z^{(3)})$$



60

60

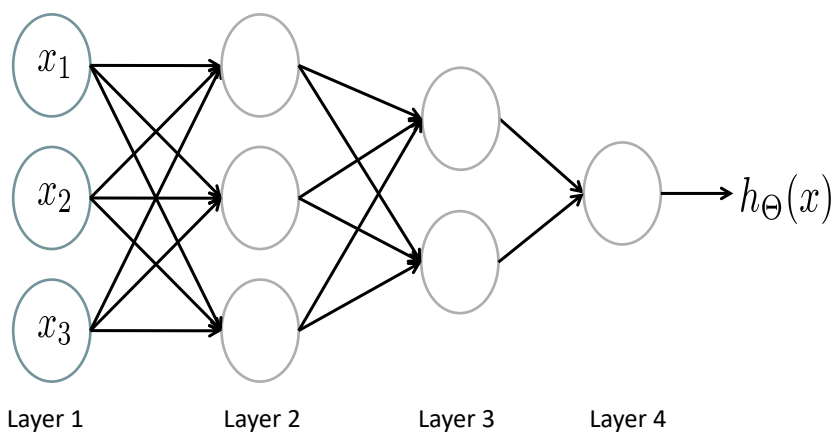
Neural Network learning its own features



61

61

Other network architectures



62

62

EXAMPLES AND INTUITIONS I

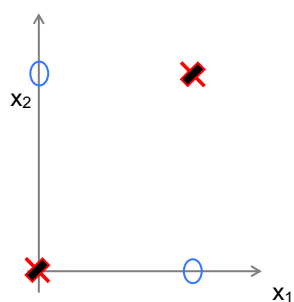


63

63

Non-linear classification example: XOR/XNOR

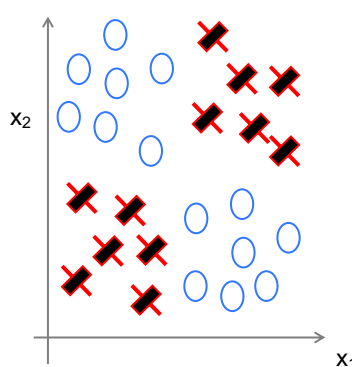
x_1, x_2 are binary (0 or 1).



$$y = x_1 \text{ XOR } x_2$$

$$x_1 \text{ XNOR } x_2$$

$$\text{NOT } (x_1 \text{ XOR } x_2)$$



64

64

Simple example: AND

$x_1, x_2 \in \{0, 1\}$
 $y = x_1 \text{ AND } x_2$

x_1 x_2 $h_{\Theta}(x)$

$g(z)$

x_1	x_2	$h_{\Theta}(x)$
0	0	
0	1	
1	0	
1	1	

65

65

Example: OR function

$h_{\Theta}(x)$

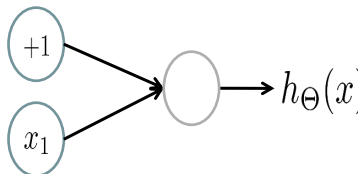
x_1	x_2	$h_{\Theta}(x)$
0	0	
0	1	
1	0	
1	1	

66

66

x_1 AND x_2
 x_1 OR x_2

Negation:



```

graph LR
    A((+1)) --> C(( ))
    B((x1)) --> C
    C --> D(h_theta(x))
            
```

x_1	$h_{\Theta}(x)$
0	
1	

$$h_{\Theta}(x) = g(10 - 20x_1)$$

$$(\text{NOT } x_1) \text{ AND } (\text{NOT } x_2)$$

67

Putting it together: x_1 XNOR x_2

Diagram illustrating the implementation of the AND operation using a single neuron. The inputs are $+1$, x_1 , and x_2 . The weights are -30 for $+1$, 20 for x_1 , and 20 for x_2 . The output is $h_{\Theta}(x)$.

x_1 AND x_2

Diagram illustrating the implementation of the operation $(\text{NOT } x_1) \text{ AND } (\text{NOT } x_2)$ using a single neuron. The inputs are $+1$, x_1 , and x_2 . The weights are 10 for $+1$, -20 for x_1 , and -20 for x_2 . The output is $h_{\Theta}(x)$.

$(\text{NOT } x_1) \text{ AND } (\text{NOT } x_2)$

Diagram illustrating the implementation of the OR operation using a single neuron. The inputs are $+1$, x_1 , and x_2 . The weights are -10 for $+1$, 20 for x_1 , and 20 for x_2 . The output is $h_{\Theta}(x)$.

x_1 OR x_2

Diagram illustrating the implementation of the XNOR operation using a neural network. The input layer has nodes $+1$, x_1 , and x_2 . The hidden layer has nodes $a_1^{(1)}$ and $a_2^{(1)}$. The output layer has node $a_1^{(2)}$. The weights are: -30 from $+1$ to $a_1^{(1)}$, 20 from x_1 to $a_1^{(1)}$, 20 from x_2 to $a_1^{(1)}$, 10 from $+1$ to $a_2^{(1)}$, -20 from x_1 to $a_2^{(1)}$, -20 from x_2 to $a_2^{(1)}$, -10 from $+1$ to $a_1^{(2)}$, 20 from $a_1^{(1)}$ to $a_1^{(2)}$, and 20 from $a_2^{(1)}$ to $a_1^{(2)}$. The output is $h_{\Theta}(x)$.

x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_{\Theta}(x)$
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

68

68

Non-linear classification example: XOR/XNOR

x_1, x_2 are binary (0 or 1).

x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_{\Theta}(x)$
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1



69

Neural Network intuition

Layer 1 Layer 2 Layer 3 Layer 4



70

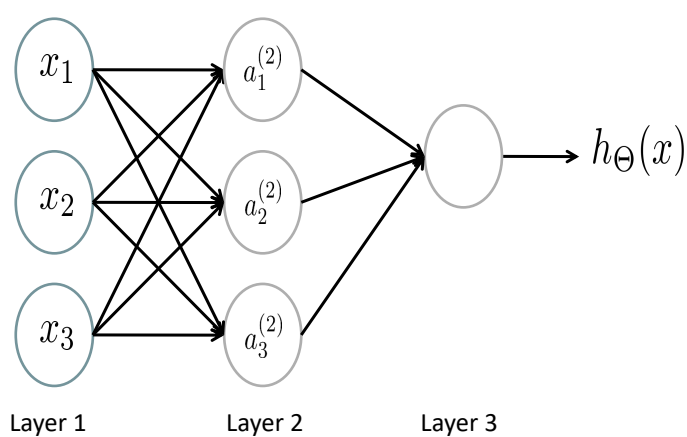
NEURAL NETWORKS



71

71

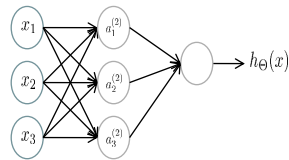
Neural Network learning its own features



72

72

Forward propagation: Vectorized Implementation



$$\begin{aligned}
 a_1^{(2)} &= g(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3) \\
 a_2^{(2)} &= g(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3) \\
 a_3^{(2)} &= g(\theta_{30}^{(1)} x_0 + \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{33}^{(1)} x_3) \\
 h_{\Theta}(x) &= g(\theta_{10}^{(2)} a_0^{(2)} + \theta_{11}^{(2)} a_1^{(2)} + \theta_{12}^{(2)} a_2^{(2)} + \theta_{13}^{(2)} a_3^{(2)})
 \end{aligned}$$

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix}$$

$$z^{(2)} = \Theta^{(1)} x$$

$$a^{(2)} = g(z^{(2)})$$

$$\text{Add } a_0^{(2)} = 1.$$

$$z^{(3)} = \Theta^{(2)} a^{(2)}$$

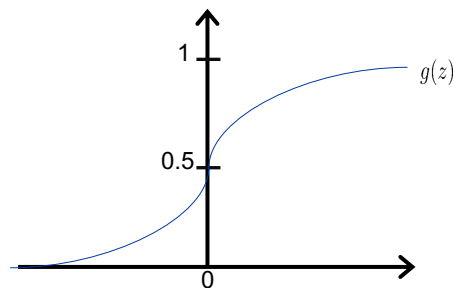
$$h_{\Theta}(x) = a^{(3)} = g(z^{(3)})$$



73

73

- $h(\theta)$ is a sigmoid function



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

74

74

MULTI-CLASS NEURAL NETWORKS



75

75

Multi Class Problem



Pedestrian



Car



Motorcycle



Truck

$$h_{\theta}^{(1)}(x)$$

$$h_{\theta}^{(2)}(x)$$

$$h_{\theta}^{(3)}(x)$$

$$h_{\theta}^{(4)}(x)$$

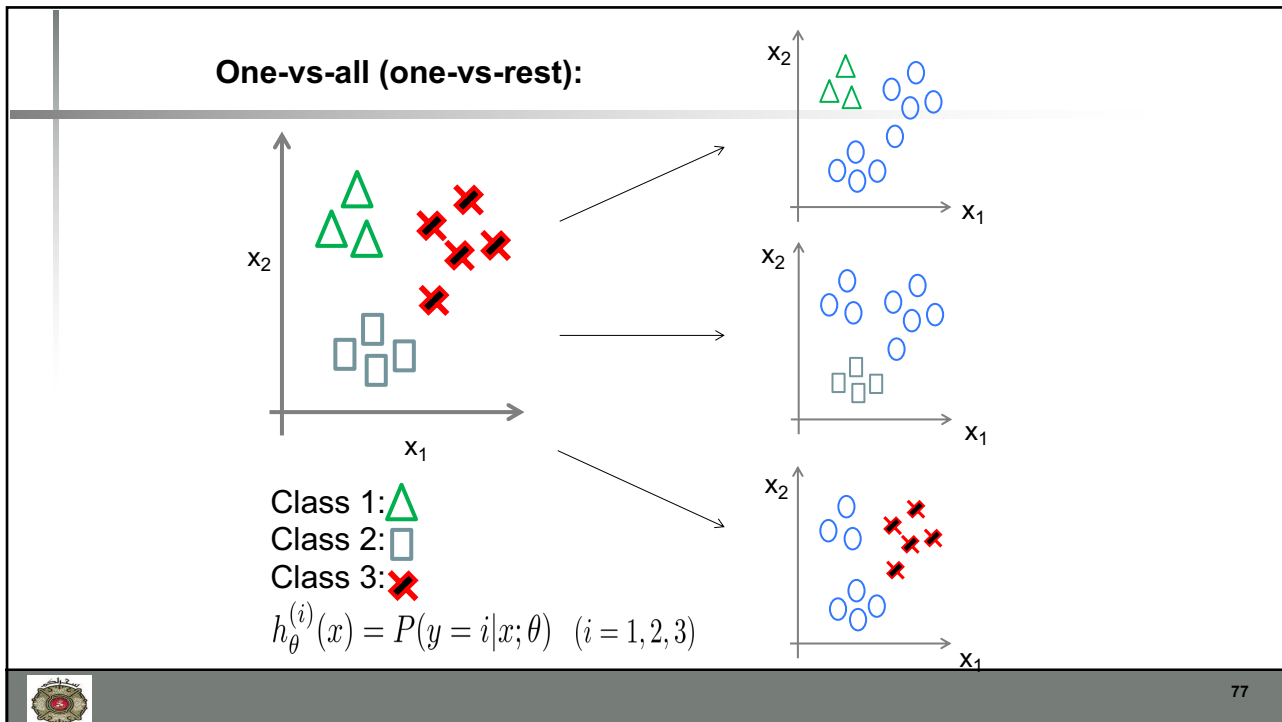
$$\max_i h_{\theta}^{(i)}(x)$$



Dr. Ali Hassan, DC&SE, College of E&ME, NUST

76

76



77

$$h_{\theta}^{(1)}(x)$$

$$h_{\theta}^{(2)}(x)$$

$$h_{\theta}^{(3)}(x)$$

$$h_{\theta}^{(4)}(x)$$

$$\max_i h_{\theta}^{(i)}(x)$$

78

Multiple output units: One-vs-all.



Pedestrian



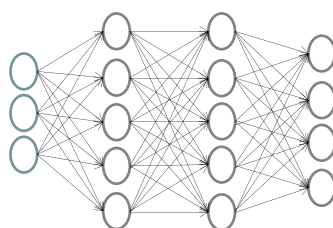
Car



Motorcycle



Truck



$$h_{\theta}(x) \in \mathbb{R}^4$$

$$\text{Want } h_{\theta}(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad h_{\theta}(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad h_{\theta}(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad \text{etc.}$$

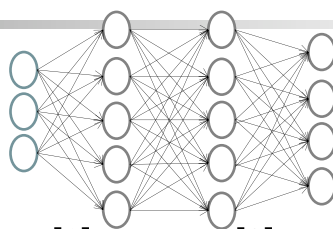
when pedestrian when car when motorcycle



79

79

Multiple output units: One-vs-all.



$$h_{\theta}(x) \in \mathbb{R}^4$$

$$\text{Want } h_{\theta}(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad h_{\theta}(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad h_{\theta}(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad \text{etc.}$$

when pedestrian when car when motorcycle

Training set: $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})$

$$y^{(i)} \text{ one of } \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

pedestrian car motorcycle truck



80

80

Neural Network (Classification)

Layer 1 Layer 2 Layer 3 Layer 4

Binary classification

$y = 0 \text{ or } 1$

1 output unit

$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

$L =$ total no. of layers in network

$s_l =$ no. of units (not counting bias unit) in layer l

Multi-class classification (K classes)

$y \in \mathbb{R}^K$ E.g. $\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$

pedestrian car motorcycle truck

K output units

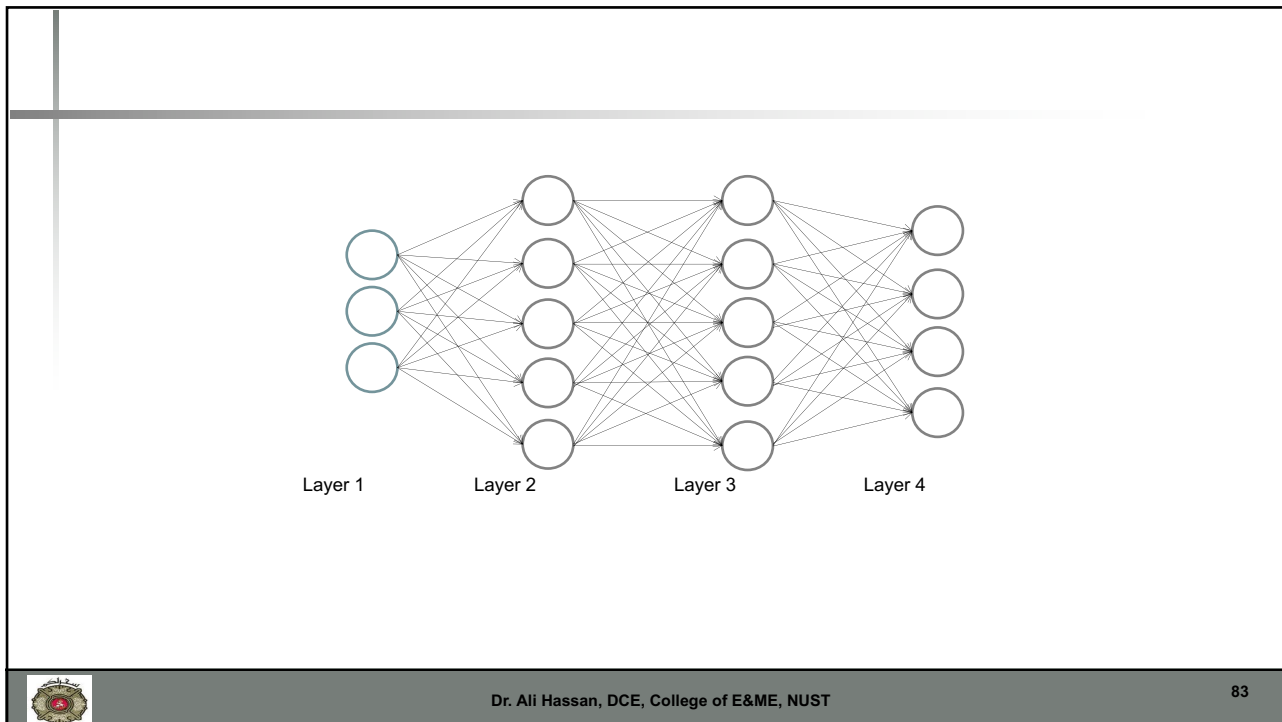
81

81

NEURAL NETWORK COST FUNCTION

82

82



83

Cost function

Logistic regression:

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

Neural network:

$$h_{\Theta}(x) \in \mathbb{R}^K \quad (h_{\Theta}(x))_i = i^{th} \text{ output}$$

$$J(\Theta) = -\frac{1}{m} \left[\sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(h_{\Theta}(x^{(i)}))_k + (1 - y_k^{(i)}) \log(1 - (h_{\Theta}(x^{(i)}))_k) \right]$$

$$+ \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\theta_{ji}^{(l)})^2$$

84

What This means ...

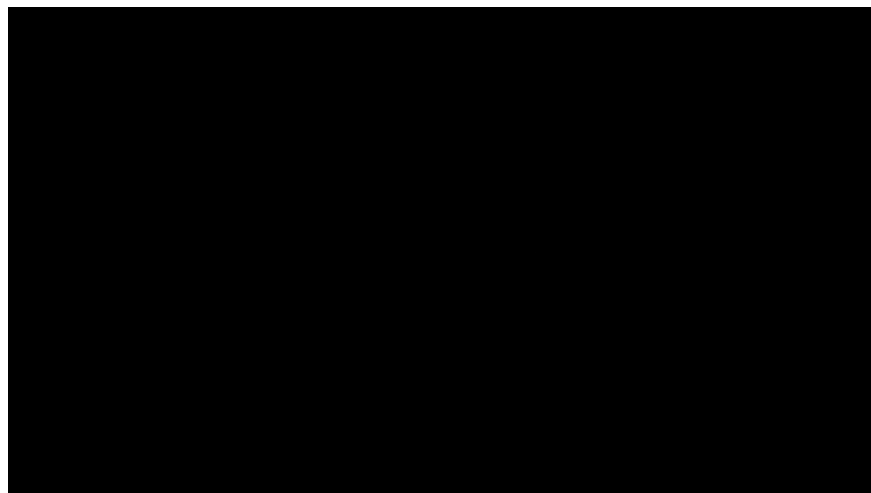
- Take the predicted output $(h_{\theta}(x^i))_k$
- Take the actual output y
- Diff of the two $h-y$
- Sum over all training examples
- That is your average cost



85

85

<https://youtu.be/IHZwWFHWa-w>



Dr. Ali Hassan, DCE, College of E&ME, NUST

86

86

Next Step – Gradient Descent

- Take derivative of the cost function $J(\theta)$ w.r.t. weights and take step in the direction of steepest descent
- i.e., direction that reduces the cost function



Dr. Ali Hassan, DCE, College of E&ME, NUST

87

87

Gradient computation

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log h_{\theta}(x^{(i)})_k + (1 - y_k^{(i)}) \log(1 - h_{\theta}(x^{(i)})_k) \right] \\ + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\theta_j^{(l)})^2$$

$$\min_{\theta} J(\theta)$$

Use Gradient Descent

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

Need code to compute:

$$\begin{aligned} &- J(\theta) \\ &- \frac{\partial}{\partial \theta_{ij}^{(l)}} J(\theta) \end{aligned}$$



88

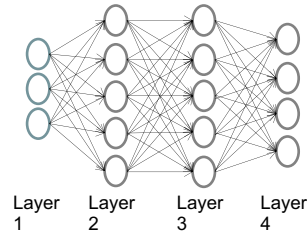
88

Forward Propagation

Given one training example (x, y) :

Forward propagation:

$$\begin{aligned} a^{(1)} &= x \\ z^{(2)} &= \Theta^{(1)} a^{(1)} \\ a^{(2)} &= g(z^{(2)}) \quad (\text{add } a_0^{(2)}) \\ z^{(3)} &= \Theta^{(2)} a^{(2)} \\ a^{(3)} &= g(z^{(3)}) \quad (\text{add } a_0^{(3)}) \\ z^{(4)} &= \Theta^{(3)} a^{(3)} \\ a^{(4)} &= h_{\Theta}(x) = g(z^{(4)}) \end{aligned}$$



89

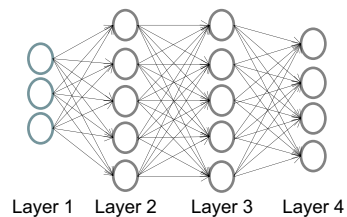
89

Gradient computation: Backpropagation algorithm

Intuition: $\delta_j^{(l)}$ = “error” of node j in layer l .

For one example (x, y)

For each output unit (layer $L = 4$)



$$\delta_j^{(4)} = a_j^{(4)} - y_j$$

$$\delta^{(3)} = (\Theta^{(3)})^T \delta^{(4)} \cdot g'(z^{(3)}) \quad g'(z^{(3)}) = (a^{(3)} \cdot (1 - a^{(3)}))$$

$$\delta^{(2)} = (\Theta^{(2)})^T \delta^{(3)} \cdot g'(z^{(2)}) \quad g'(z^{(2)}) = (a^{(2)} \cdot (1 - a^{(2)}))$$

$$\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta) = a_j^{(l)} \delta_i^{(l+1)}$$



90

90

Backpropagation algorithm

Training set $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$

Set $\Delta_{ij}^{(l)} = 0$ (for all l, i, j).

For $i = 1$ to m {

Set $a^{(1)} = x^{(i)}$

Perform forward propagation to compute $a^{(l)}$ for $l = 2, 3, \dots, L$

Using $y^{(i)}$, compute $\delta^{(L)} = a^{(L)} - y^{(i)}$

Compute $\delta^{(L-1)}, \delta^{(L-2)}, \dots, \delta^{(2)}$

$\Delta_{ij}^{(l)} := \Delta_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)} \leftarrow \Delta^{(l)} := \Delta^{(l)} + \delta^{(l+1)} (a^{(l)})^T$

$$D_{ij}^{(l)} := \frac{1}{m} \Delta_{ij}^{(l)} + \lambda \Theta_{ij}^{(l)} \text{ if } j \neq 0$$

$$D_{ij}^{(l)} := \frac{1}{m} \Delta_{ij}^{(l)} \text{ if } j = 0$$

$$\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta) = D_{ij}^{(l)}$$



91

91

Neural Networks Learning

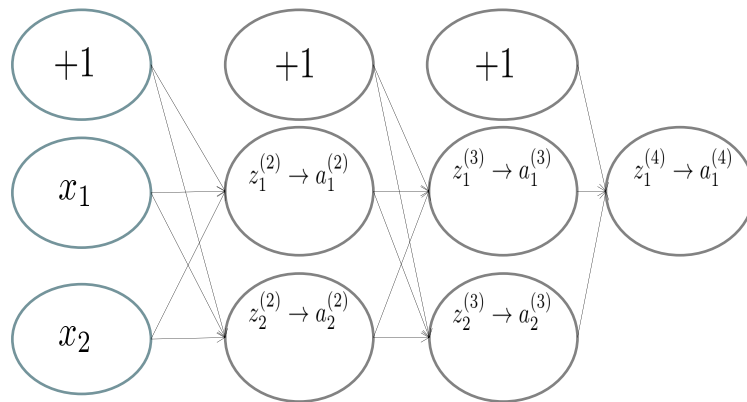
BACK-PROPAGATION INTUITION



92

92

Back Propagation



93

93

RANDOM INITIALIZATION



94

94

Initial value of Θ

For gradient descent and advanced optimization method, need initial value for Θ .

Consider gradient descent

`initialTheta = zeros(n,1)?`

Is this going to work in neural networks?

What is the issue?

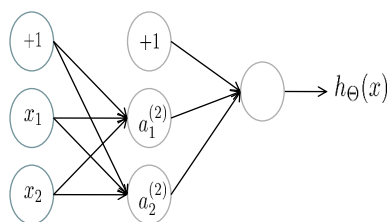


95

95

Zero initialization

$$\Theta_{ij}^{(l)} = 0 \text{ for all } i, j, l.$$



After each update, parameters corresponding to inputs going into each of two hidden units are identical.

Random initialization Works much better



96

96

SUMMARY

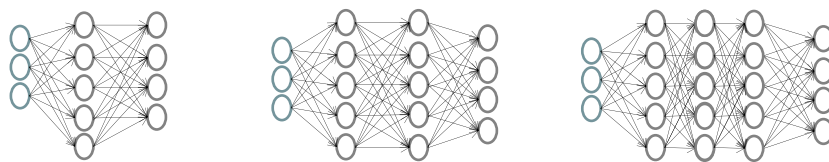


97

97

Training a neural network

Pick a network architecture (connectivity pattern between neurons)



No. of input units: Dimension of features $x^{(i)}$

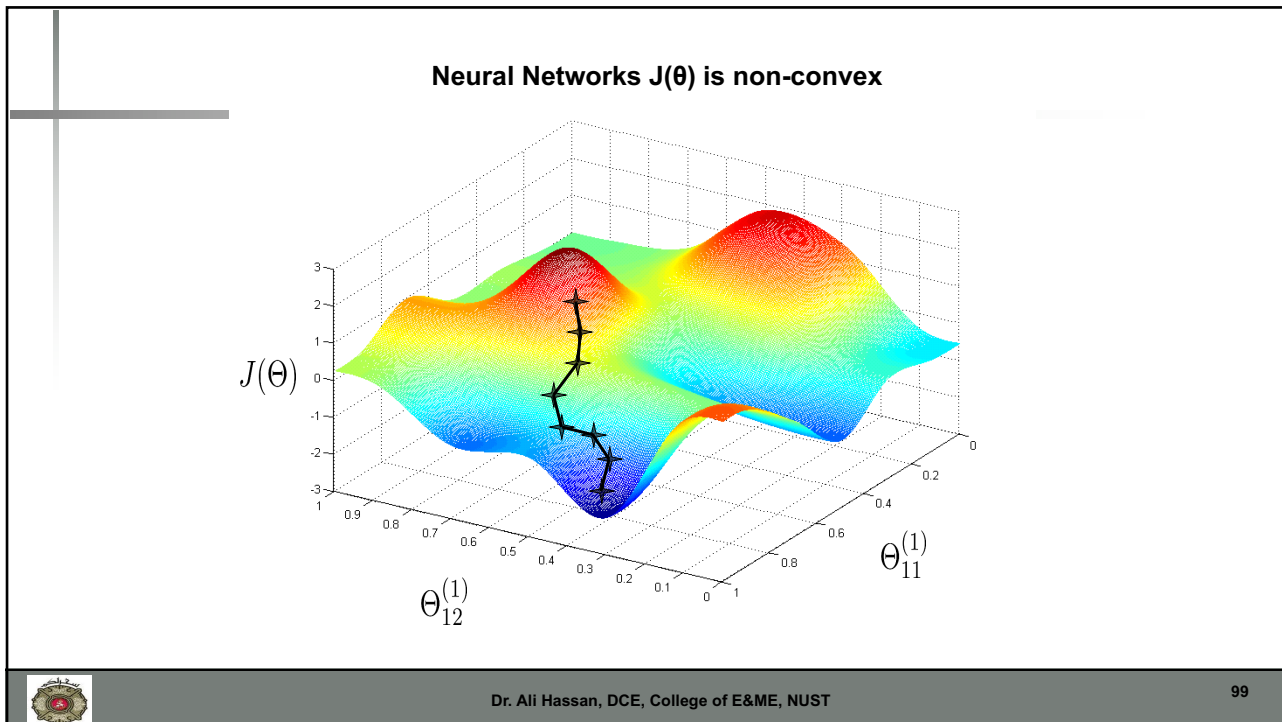
No. output units: Number of classes

Reasonable default: 1 hidden layer, or if >1 hidden layer, have same no. of hidden units in every layer (usually the more the better)



98

98



99

Python Example

```

MyXORNN.py
1  # In this code I am going to write my first NN in Python. I am going to use the code from Intern
2
3
4  import numpy as np
5  ...
12 def sigmoid(x, deriv=False):
13     if(deriv==True):
14         return (x*(1-x))
15
16     return 1/(1+np.exp(-x))
17
18
19 # We are going to generate the input data, and remember the x0 column
20
21 #input_data
22 X = np.array([[0, 0],
23              [0, 1],
24              [1, 0],
25              [1, 1]])

```



Dr. Ali Hassan, DCE, College of E&ME, NUST

100

100

Reading Material

- Lecture note by Dr Ali Hassan
- Neural Network Tutorial
- You Tube Videos
 - <https://youtu.be/aircAruvnKk>
 - [Andrew Ng Lectures](#)
 - <https://youtu.be/1ZhtwInuOD0>
- **Neural Networks and Deep Learning by Michael Nielson**
 - <http://neuralnetworksanddeeplearning.com/>

